

Chapter 4 - Objectives

- Computer Bus
- Memory Organization
- Interrupts
- Marie assembly language

- A bus is a set of wires that acts as a shared but common Datapath to connect multiple subsystems within the system. It consists of multiple lines, allowing the parallel movement of bits
- Buses are low cost but very versatile, and they make it easy to connect new devices to each other and to the system
- At any one time, only one device (be it a register, the ALU, memory, or some other component) may use the bus
- However, this sharing often results in a communications bottleneck
- **The speed of the bus is affected by its length as well as by the number of devices sharing it**
- Quite often, devices are divided into master and slave categories

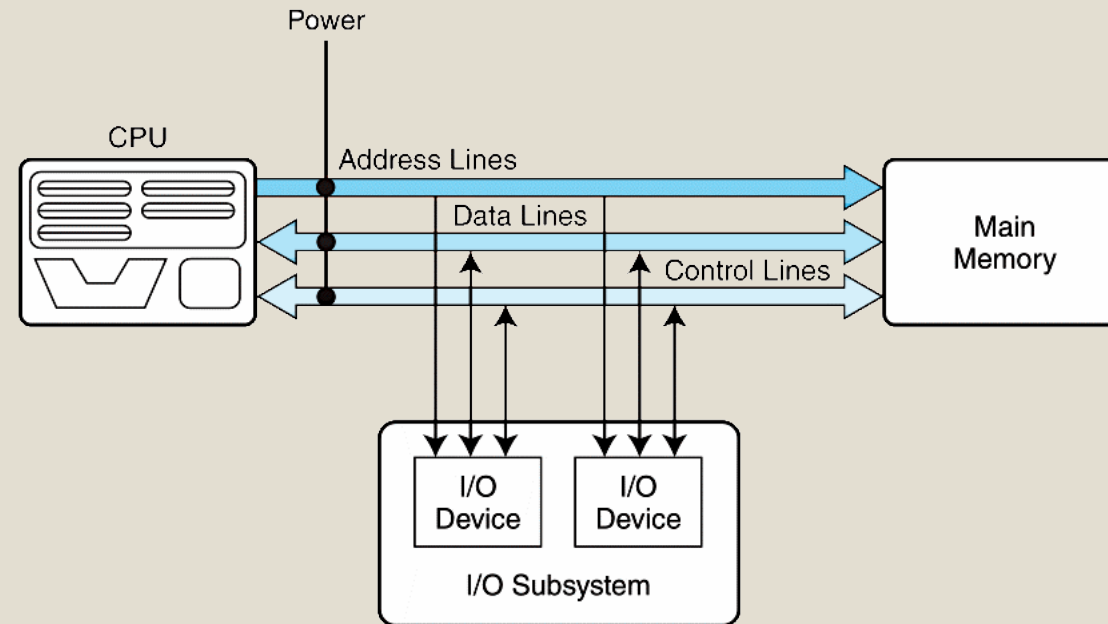
The Bus

- Buses consist of:
 - Data lines
 - Control lines
 - Address lines.
- Data lines convey bits from one device to another
- Control lines determine the direction of data flow, and when each device can access the bus.
- Address lines determine the location of the source or destination of the data.

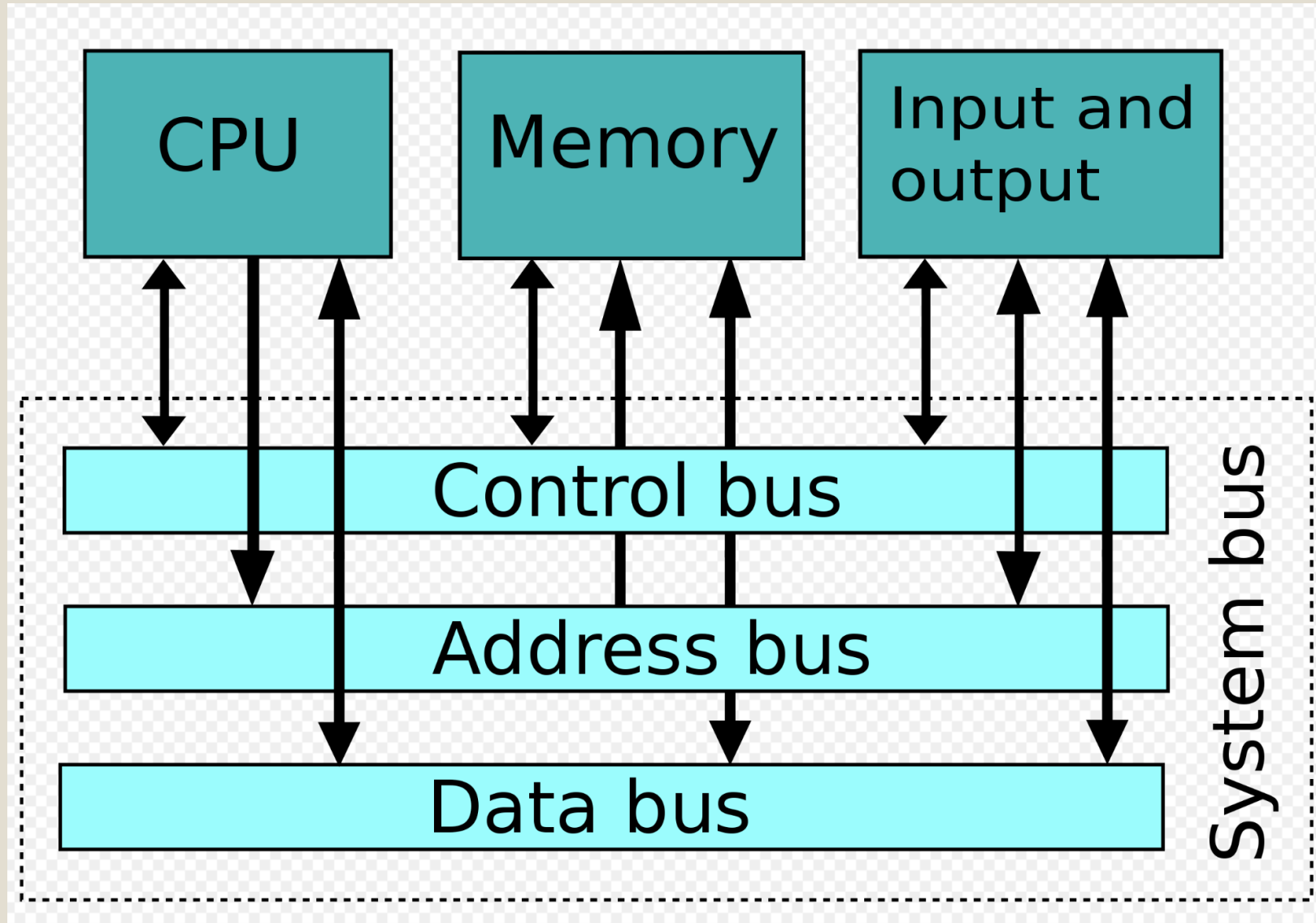
Bus

- The CPU shares data with other system components by way of a **data bus**.
 - A bus is a set of wires that simultaneously convey a single bit along each line.

The computer Buses



The computer Buses



Clocks

- Every computer contains at least one clock that synchronizes the activities of its components.
- A fixed number of clock cycles are required to carry out each data movement or computational operation.
- The clock frequency, measured in megahertz or gigahertz, determines the speed with which all operations are carried out.
- Clock cycle time is the reciprocal of clock frequency.
 - An 800 MHz clock has a cycle time of 1.25 ns.

Clock cycle

- A computer processor or **CPU** speed is determined by the **clock cycle**, which is the amount of time between two pulses of an **oscillator**.
- Generally speaking, the higher number of pulses per second, the faster the computer processor can process information.
- The **clock speed** is measured in Hz, often either megahertz (**MHz**) or gigahertz (**GHz**). For example, a 4 GHz processor performs 4,000,000,000 clock cycles per second.
- Computer processors can execute one or more instructions per clock cycle, depending on the type of processor.
-
- Early computer processors and slower CPUs can only execute one instruction per clock cycle, but modern processors can execute multiple instructions per clock cycle.

Clocks

- Clock speed should not be confused with CPU performance.
- The CPU time required to run a program is given by the general performance equation:
- We see that we can improve CPU throughput when we reduce the number of instructions in a program, reduce the number of cycles per instruction, or reduce the number of nanoseconds per clock cycle.

$$\text{CPU Time} = \frac{\text{seconds}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \times \frac{\text{avg. cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}}$$

CPU Time

Number of Instructions in program			5,000,000
Instruction	Frequency	INSTRUCTION Cycle Per Instruction	Cycle Per Instruction(CPI)
I	F	I-CPI	F * I-CPI
ALU	50%	3	1.5
Load	20%	5	1
Store	10%	4	0.4
branch	20%	3	0.6
		SUM PROGRAM CYCLE PER INSTRUCTION	3.5

CPU Time

Number of Instructions in program	AVG CYCLES PER INSTRUCTION	Cycle time = duration of clock cycle	Clock Frequency(F)		CPU TIME = (I * CPI) / F	
I	CPI	CLOCK RATE (GHZ)	F	=	CPU TIME(SECONDS)	CPU TIME(MILI-SECONDS)
5,000,000	3.5	2	$2 * 10^9$	2,000,000,000	0.00875	8.75
5,000,000	3.5	2.5	$2.5 * 10^9$	2,500,000,000	0.00700	7.00
5,000,000	3.5	3.0	$3 * 10^9$	3,000,000,000	0.00583	5.83
5,000,000	3.5	4.0	$4 * 10^9$	4,000,000,000	0.00438	4.38

Interrupts ?

- An interrupt is **a signal emitted by a device attached to a computer or from a program within the computer**
- Interrupts are **signals sent to the CPU by external devices**, normally I/O devices
- It requires the operating system (OS) to stop and figure out what to do next. An interrupt temporarily stops or terminates a service or a current process
- They tell the CPU to stop its current activities and execute the appropriate part of the operating system
- Interrupts are events that alter (or interrupt) the normal flow of execution in the system
- Hardware Interrupts are generated by hardware devices to signal that they need some attention from the OS
- In computer systems programming, an **interrupt handler**, also known as an interrupt service routine or ISR, is **a special block of code associated with a specific interrupt condition**

Interrupts ?

◦ **Interrupts** are events that alter (or interrupt) the normal flow of execution in the system. An interrupt can be triggered for a variety of reasons, including:

◦ I/O requests

◦ Arithmetic errors (e.g., division by 0)

◦ Arithmetic underflow or overflow

◦ Hardware malfunction (e.g., memory parity error)

◦ User-defined break points (such as when debugging a program)

◦ Page faults (this is covered in detail in Chapter 6)

◦ Invalid instructions (usually resulting from pointer issues)

◦ Miscellaneous

Programming Languages

- High Level language(C++, Java, Python, C#)
- Assembly Language
- Machine Language

High Level Language

- HLL (High Level Language) programs are machine independent
- They are easy to learn, easy to use, and convenient for managing complex tasks
- It enables a user to write a program in a language that resembles English words and familiar mathematical symbols,
- Examples of high-level language are python, c++, Java

Assembly language

- Assembly language programs are machine specific
- It is the language that the processor directly understands
- It is a low-level language that allows users to write a program using alphanumeric mnemonic codes

HIGH LEVEL VS ASSEMBLY LANGUAGE

High level language:

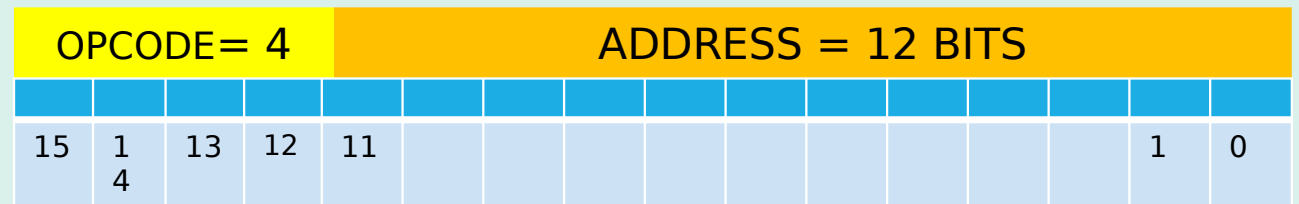
$$Z = X \times Y + W \times U$$

In assembly language it looks like this:

```
LOAD X
MULT Y
STORE TEMP
LOAD W
MULT U
ADD TEMP
STORE Z
```

MARIE

- The MARIE architecture has the following characteristics:
 - Binary, two's complement data representation.
 - Stored program, fixed word length data and instructions.
 - 4K words of word-addressable main memory. ($4 * 1024 = 4096$ Bytes)
 - 16-bit data words
 - 16-bit instructions:
 - 4 for the opcode
 - 12 for the address.
 - A 16-bit arithmetic logic unit (ALU).
 - Seven registers for control and data movement.

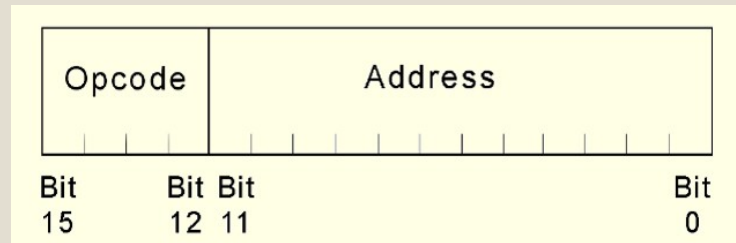


Marie machine instruction

1. **OPCODE:** The opcodes specify the operations to be executed
2. **OPERAND:** The operands specify register or memory locations of data

[illegible]

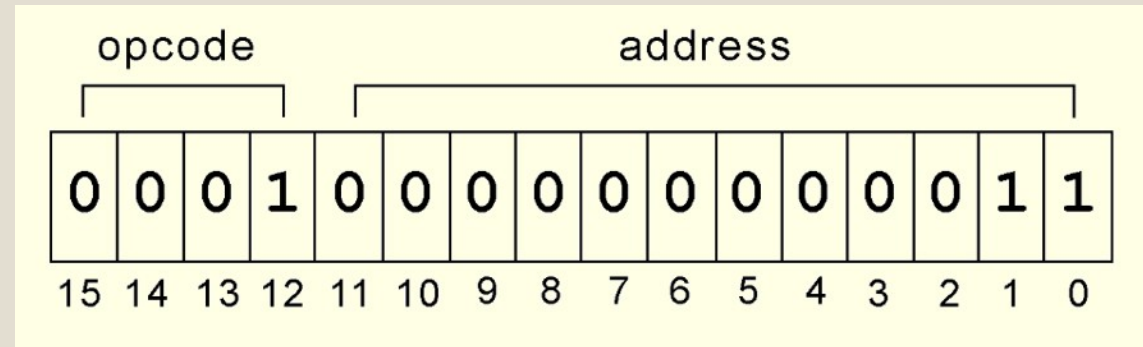
MARIE(the format of a MARIE instruction)



Instruction Number			
Binary	Hex	Instruction	Meaning
0001	1	Load X	Load contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC.
0100	4	Subt X	Subtract the contents of address X from AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate program.
1000	8	Skipcond	Skip next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MARIE

- Load 003
- This is a bit pattern for a **LOAD** instruction as it would appear in the IR:



- We see that the opcode is 1 and the address from which to load the data is 3.

MARIE(the format of a MARIE instruction)

Check MarieGuide.pdf file
in chapter 4 Resources
folder